

Classi astratte

Introduzione

Nel paradigma della programmazione orientata agli oggetti (OOP), le **classi astratte** rappresentano un concetto fondamentale per progettare architetture flessibili, riutilizzabili e ben strutturate.

Queste classi non vengono istanziate direttamente, ma servono come base per altre classi che dovranno implementare determinati comportamenti.

Cosa sono le classi astratte?

Una **classe astratta** è una classe che:

- **non può essere istanziata direttamente** ;
- **può contenere uno o più metodi astratti, ovvero metodi che sono dichiarati ma non hanno un corpo** ;
- **serve da modello per le classi derivate (sottoclassi), che sono obbligate a implementare i metodi astratti dichiarati** .

Il modulo `abc`

In Python, per definire una classe astratta si usa il modulo `abc` , e si eredita da `ABC` (Abstract Base Class). I metodi astratti si definiscono usando il decoratore `@abstractmethod` .

Per dirla meglio il modulo `abc` , fornisce due elementi essenziali:

- `ABC` : **la classe base da cui devono ereditare le classi astratte.**
- `@abstractmethod` : **un decoratore per definire metodi astratti, ossia metodi che devono essere implementati dalle sottoclassi.**

si scrive così :



Python

```
1 from abc import ABC, abstractmethod
```

✓ Vantaggi della classi astratte

- Forzano le **sottoclassi** a implementare certi metodi, garantendo una **struttura comune**.

- Rendono il codice più **modulare**, **manutenibile** e **polimorfico**.
- Ideali per rappresentare **concetti generici** che avranno comportamenti diversi nelle specifiche implementazioni.

Esempio concreto di implementazione delle classi astratte

Step 1: creazione file `FormaGenerica.py`

All'interno di questo file creiamo una classe generica chiamata `FormaGenerica` pensata per essere la **superclasse** di altre classi come `Rettangolo`, `Triangolo`, ecc.

Questa classe deve **imporre l'esistenza** del metodo `draw()` nelle classi figlie, ma **senza implementarlo** direttamente.



Python

```
1  from abc import ABC, abstractmethod
2
3  class FormaGenerica(ABC):
4      def __init__(self, shape: str) -> None:
5          if not shape:
6              raise ValueError("Shape non può essere una stringa
vuota")
7          self.shape = shape
8
9      @abstractmethod
10     def draw(self) -> None:
11         pass
12
13     def setShape(self, shape: str) -> None:
14         if shape:
15             self.shape = shape
16         else:
17             raise ValueError("Shape non può essere una stringa
vuota")
18
19     def getShape(self) -> str:
20         return self.shape
```

Analisi:

- `FormaGenerica` è una **classe astratta** che eredita da `ABC`.

- Il metodo `draw()` è **astratto**, cioè viene solo dichiarato ma **non implementato**: ogni classe derivata dovrà fornirne una definizione concreta.
- Sono presenti anche due metodi **concreti** (`setShape` e `getShape`) per gestire il nome della forma. Questi possono essere già usati dalle sottoclassi.

Step 2: creazione file `rettangolo.py`

Questo file serve per implementare il metodo `draw()` in una classe specifica che andrà a disegnare la figura geometrica di un rettangolo:



Python

```

1  from formaGenerica import FormaGenerica
2
3  class Rettangolo(FormaGenerica):
4      def __init__(self):
5          super().__init__()
6          self.setShape("rettangolo")
7
8      def draw(self) -> None:
9          print(f"\n{self.getShape()}\n")
10         for i in range(5):
11             for j in range(10):
12                 if i == 0 or i == 4 or j == 0 or j == 9:
13                     print("*", end="")
14                 else:
15                     print(" ", end="")
16             print()

```

Analisi:

- La classe `Rettangolo` eredita da `FormaGenerica` e implementa il metodo astratto `draw()`.
- Il metodo `draw()` stampa un rettangolo di **5 righe x 10 colonne** usando `*`. Usa due cicli annidati per disegnare:
- Le **righe superiori e inferiori** (`i == 0` o `i == 4`)
- Le **colonne laterali** (`j == 0` o `j == 9`)
- Il resto viene lasciato **vuoto** per dare l'effetto "bordo".
- Grazie al metodo `setShape()` della classe astratta, possiamo indicare il tipo di forma senza riscrivere il codice.

Step 3: Creazione file `test_astratto.py`

Questo file serve per testare quanto fatto finora attraverso le istanze della classe `Rettangolo`



Python

```
1 from rettangolo import Rettangolo
2
3 r: Rettangolo = Rettangolo()
4 r.draw()
5
```

Analisi:

- In questo file testiamo la nostra classe concreta `Rettangolo` .
- Se si provasse a creare un oggetto direttamente dalla classe `FormaGenerica` , si otterrebbe un errore perché non è stata fornita un'implementazione per il metodo `draw()` .
- Il codice funziona correttamente solo dopo che `Rettangolo` implementa completamente i metodi astratti ereditati.

Quindi le classi astratte sono uno strumento potente per definire comportamenti obbligatori nelle sottoclassi, migliorando la coerenza del codice e sfruttando il **polimorfismo**. In questo esempio, `FormaGenerica` agisce come blueprint per tutte le forme, mentre `Rettangolo` fornisce una specifica implementazione del disegno.

📜 Regola d'oro:

se una classe contiene almeno un metodo astratto, allora è astratta e non può essere istanziata direttamente !